



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Цифрова схемотехніка

Домашня контрольна робота

«Розроблення модулів арифметико-логічних блоків»

Виконали: студенти 2 курсу ФІОТ, гр. ІО-41

Давидчук А. М.

Дубров Д. О.

Залікова книжка № 4106

Перевірив: *Нікольський С. С.*

Тема: «Розроблення модулів арифметико-логічних блоків».

Мета роботи: розробити та дослідити роботу арифметико-логічного блока із загальними мікроопераціями, реалізувати формування ознак результату, виконати функціональне моделювання за допомогою тестового рівня коду та додатково реалізувати нормалізацію результату шляхом зсуву праворуч на один розряд.

Хід роботи

Визначення варіанта

Номер залікової книжки: 4106. У двійковому вигляді:

$$4106_{10} = 0001\ 0000\ 0000\ 1010_2.$$

Звідси:

$$h_7 = 0, h_6 = 0, h_5 = 0, h_4 = 1, h_3 = 0, h_2 = 1, h_1 = 0.$$

За таблицею вибору варіанта для розроблення АЛБ для набору

$$h_4 = 1, h_2 = 1, h_3 = 0, h_1 = 0$$

отримано такий набір мікрооперацій:

ALB_MI I_4I_3	Мікрооперація
00	$R \vee S$
01	$R + S + CI$
10	$S - R - 1 + CI$
11	$R \vee S$

Табл. 1: Мікрооперації АЛБ за варіантом

Структура проєкту

Проєкт реалізовано у каталозі `Davydchuk_Dubrov_Mega_Lab_ALB`. Для моделювання використано підхід з окремим тестовим рівнем, який складається з генератора вхідних параметрів, базової моделі та модуля перевірки.

Файл	Призначення
<code>src/alb.v</code>	основний модуль АЛБ із вхідними регістрами, формуванням результату, ознак та нормалізованого виходу
<code>sim/alb_ref.v</code>	еталонна математична модель для порівняння
<code>sim/stimulus.v</code>	генератор тестових векторів
<code>sim/checker.v</code>	модуль автоматичної перевірки результатів
<code>sim/top.v</code>	верхній модуль тестового рівня
<code>sim/run.do</code>	скрипт запуску компіляції та симуляції в ModelSim

Табл. 2: Склад проєкту

Модуль параметризовано за розрядністю `WIDTH`. У тестовому рівні використано `WIDTH = 12`, щоб дослідити приклади з розрядністю, достатньою для знакового результату і нормалізації.

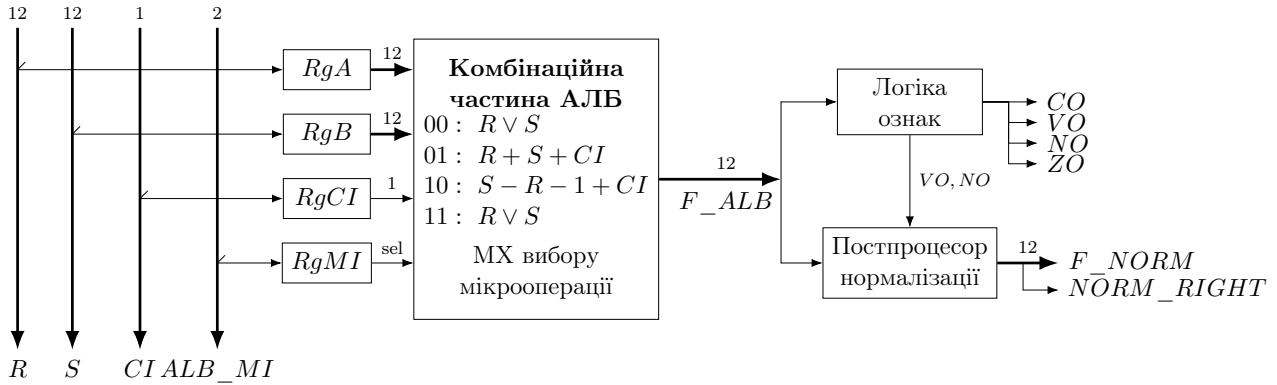


Рис. 1: Структурна схема розробленого АЛБ

Розподіл виконання роботи

Роботу виконано спільно. Основну функціональну частину проєкту, тобто визначення варіанта, розроблення структури АЛБ, реалізацію мікрооперацій та формування ознак результату, виконав Давидчук А. М. Дубров Д. О. виконав тестову частину роботи: підготовку тестових векторів, еталонної моделі, автоматичної перевірки результатів, запуск функціонального моделювання в ModelSim та оформлення результатів у звіті.

Реалізація АЛБ

Вхідні аргументи `A_ALB`, `B_ALB`, перенос `CI` та мікроінструкція `ALB_MI` записуються у буферні регістри за фронтом `clock`. При активному нулі `resetb` регістри очищуються.

Для мікрооперації 10 вираз

$$S - R - 1 + CI$$

реалізовано через доповняльний код:

$$S - R - 1 + CI = S + \overline{R} + CI.$$

Ознаки формуються так:

- `CO` — перенос за межі розрядної сітки;
- `VO` — переповнення для арифметичних операцій у доповняльному коді;
- `NO` — знак результату, тобто старший біт;
- `ZO` — рівність результату нулю;
- `NORM_RIGHT` — необхідність нормалізації праворуч.

Повну реалізацію вибору мікрооперацій наведено у лістингу файлу `src/alb.v`. У звіті цей код підтягується без дублювання вручну через команду `\inputminted`.

Додаткове завдання реалізовано як постпроцесор. Якщо встановлено ознаку переповнення і порушено нормалізацію ліворуч, результат зсувається на один розряд праворуч зі збереженням знакового біта:

Повний код постпроцесора також наведено у лістингу `src/alb.v`.

Тестовий рівень

Тестовий рівень побудовано за структурою:

Stimulus → DUT || Reference model → Checker.

`stimulus.v` подає на вхід АЛБ різні комбінації `ALB_MI`, аргументів та переносу `CI`. `alb_ref.v` обчислює очікуваний результат математично. `checker.v` порівнює результат основного модуля з еталонною моделлю.

Команда запуску симуляції в ModelSim:

```
1 cd Davydchuk_Dubrov_Mega_Lab_ALB/sim
2 vsim -do run.do
```

Для консольного запуску:

```
1 vsim -c -do run.do
```

Результати моделювання

Під час моделювання перевірено логічні операції, додавання, віднімання у доповняльному коді, формування ознак і роботу нормалізації. Нижче наведено фрагмент Transcript ModelSim, у якому видно компіляцію модулів, запуск верхнього рівня та проходження автоматичної перевірки:

```
1 # do run.do
2 # vlog ../src/alb.v alb_ref.v stimulus.v checker.v top.v
3 # -- Compiling module alb
4 # -- Compiling module alb_ref
5 # -- Compiling module stimulus
6 # -- Compiling module checker
7 # -- Compiling module top
8 # Top level modules:
9 #   top
10 # vsim work.top
11 # Loading work.top
12 # Loading work.alb
13 # Loading work.alb_ref
14 # Loading work.stimulus
15 # Loading work.checker
16 # run -all
17 # 80000 correct F=010000100101 FN=010000100101 C=0 V=0 N=0 Z=0 NR=0
18 # 100000 correct F=000111011010 FN=000111011010 C=1 V=0 N=0 Z=0 NR=0
19 # 110000 correct F=100100000000 FN=110010000000 C=0 V=1 N=1 Z=0 NR=1
20 # 120000 correct F=001100000000 FN=001100000000 C=1 V=1 N=0 Z=0 NR=0
21 # 130000 correct F=000000000010 FN=000000000010 C=1 V=0 N=0 Z=0 NR=0
22 # 140000 correct F=111111111110 FN=111111111110 C=0 V=0 N=1 Z=0 NR=0
23 # 170000 correct F=000000000000 FN=000000000000 C=1 V=0 N=0 Z=1 NR=0
24 # TEST PASSED
25 # ** Note: $finish      : stimulus.v(58)
26 #   Time: 245 ns  Iteration: 1  Instance: /top/stim
```


Лістинг програмного коду

Нижче наведено повний код модулів і скриптів, які використовуються для функціонального моделювання АЛБ.

- `src/alb.v` — основний модуль АЛБ. У ньому реалізовано вхідні буферні регістри, вибір мікрооперації за сигналом `ALB_MI`, формування результату `F_ALB`, ознак `CO`, `VO`, `NO`, `ZO`, а також бонусний постпроцесор нормалізації з виходами `F_NORM` і `NORM_RIGHT`.
- `sim/alb_ref.v` — еталонна математична модель. Вона описує ті самі мікрооперації без структурної деталізації АЛБ і використовується для порівняння з основним модулем.
- `sim/stimulus.v` — генератор тестових векторів. Він послідовно задає `A_ALB`, `B_ALB`, `CI` та `ALB_MI`, щоб перевірити логічні операції, додавання, віднімання, перенос, переповнення, знак, нульовий результат і нормалізацію.
- `sim/checker.v` — модуль автоматичної перевірки. Він порівнює результат основного АЛБ з еталонною моделлю та виводить `error` у разі розбіжності або `TEST PASSED`, якщо всі перевірки пройдено.
- `sim/top.v` — верхній модуль тестового рівня. Він з'єднує основний АЛБ, еталонну модель, генератор тестів і модуль перевірки, а також формує сигнали `clock` і `resetb`.
- `sim/run.do` — основний скрипт ModelSim. Він компілює файли, запускає симуляцію, додає основні сигнали на часову діаграму та виконує моделювання до завершення тесту.
- `sim/wave_pretty.do` — допоміжний скрипт для оформлення часових діаграм. Він групує сигнали, додає сигнали еталонної моделі та налаштовує вигляд вікна Wave для створення зрозумілого скріншоту.

`src/alb.v`

```
1  `timescale 1 ns / 1 ps
2
3  module alb
4  #(
5      parameter WIDTH = 12
6  )
7  (
8      output reg [WIDTH-1:0] F_ALB,
9      output reg [WIDTH-1:0] F_NORM,
10     output reg CO,
11     output reg VO,
12     output reg NO,
13     output reg ZO,
14     output reg NORM_RIGHT,
15     input [WIDTH-1:0] A_ALB,
16     input [WIDTH-1:0] B_ALB,
17     input CI,
18     input [1:0] ALB_MI,
19     input clock,
20     input resetb
```

```

21 );
22 reg [WIDTH-1:0] RgA;
23 reg [WIDTH-1:0] RgB;
24 reg RgCI;
25 reg [1:0] RgMI;
26
27 reg [WIDTH:0] ext_result;
28 reg [WIDTH-1:0] raw_result;
29 reg carry_result;
30 reg overflow_result;
31
32 always @(posedge clock or negedge resetb) begin
33     if (resetb == 1'b0) begin
34         RgA <= {WIDTH{1'b0}};
35         RgB <= {WIDTH{1'b0}};
36         RgCI <= 1'b0;
37         RgMI <= 2'b00;
38     end else begin
39         RgA <= A_ALB;
40         RgB <= B_ALB;
41         RgCI <= CI;
42         RgMI <= ALB_MI;
43     end
44 end
45
46 always @(*) begin
47     ext_result = {1'b0, RgA | RgB};
48     raw_result = RgA | RgB;
49     carry_result = 1'b0;
50     overflow_result = 1'b0;
51
52     case (RgMI)
53         2'b00: begin
54             ext_result = {1'b0, RgA | RgB};
55             raw_result = ext_result[WIDTH-1:0];
56         end
57         2'b01: begin
58             ext_result = {1'b0, RgA} + {1'b0, RgB} + RgCI;
59             raw_result = ext_result[WIDTH-1:0];
60             carry_result = ext_result[WIDTH];
61             overflow_result = (RgA[WIDTH-1] == RgB[WIDTH-1]) &&
62                             (raw_result[WIDTH-1] != RgA[WIDTH-1]);
63         end
64         2'b10: begin
65             ext_result = {1'b0, RgB} + {1'b0, ~RgA} + RgCI;
66             raw_result = ext_result[WIDTH-1:0];
67             carry_result = ext_result[WIDTH];
68             overflow_result = (RgB[WIDTH-1] != RgA[WIDTH-1]) &&
69                             (raw_result[WIDTH-1] != RgB[WIDTH-1]);
70         end
71         2'b11: begin
72             ext_result = {1'b0, RgA | RgB};
73             raw_result = ext_result[WIDTH-1:0];
74         end

```

```

75         endcase
76     end
77
78     always @(*) begin
79         F_ALB = raw_result;
80         CO = carry_result;
81         VO = overflow_result;
82         NO = raw_result[WIDTH-1];
83         ZO = (raw_result == {WIDTH{1'b0}});
84
85         NORM_RIGHT = overflow_result && (raw_result[WIDTH-1] !=
86         ↪ raw_result[WIDTH-2]);
87         if (NORM_RIGHT) begin
88             F_NORM = {raw_result[WIDTH-1], raw_result[WIDTH-1:1]};
89         end else begin
90             F_NORM = raw_result;
91         end
92     end
93 endmodule

```

sim/alb_ref.v

```

1  `timescale 1 ns / 1 ps
2
3  module alb_ref
4  #(
5      parameter WIDTH = 12
6  )
7  (
8      output reg [WIDTH-1:0] F_REF,
9      output reg [WIDTH-1:0] F_NORM_REF,
10     output reg CO_REF,
11     output reg VO_REF,
12     output reg NO_REF,
13     output reg ZO_REF,
14     output reg NORM_RIGHT_REF,
15     input [WIDTH-1:0] A_ALB,
16     input [WIDTH-1:0] B_ALB,
17     input CI,
18     input [1:0] ALB_MI,
19     input clock,
20     input resetb
21 );
22     reg [WIDTH-1:0] RgA;
23     reg [WIDTH-1:0] RgB;
24     reg RgCI;
25     reg [1:0] RgMI;
26     reg [WIDTH:0] temp;
27
28     always @(posedge clock or negedge resetb) begin
29         if (resetb == 1'b0) begin
30             RgA <= {WIDTH{1'b0}};
31             RgB <= {WIDTH{1'b0}};
32             RgCI <= 1'b0;

```



```

33         RgMI <= 2'b00;
34     end else begin
35         RgA <= A_ALB;
36         RgB <= B_ALB;
37         RgCI <= CI;
38         RgMI <= ALB_MI;
39     end
40 end
41
42 always @(*) begin
43     temp = {1'b0, RgA | RgB};
44     CO_REF = 1'b0;
45     VO_REF = 1'b0;
46
47     case (RgMI)
48         2'b00: temp = {1'b0, RgA | RgB};
49         2'b01: begin
50             temp = {1'b0, RgA} + {1'b0, RgB} + RgCI;
51             CO_REF = temp[WIDTH];
52             VO_REF = (RgA[WIDTH-1] == RgB[WIDTH-1]) &&
53                 (temp[WIDTH-1] != RgA[WIDTH-1]);
54         end
55         2'b10: begin
56             temp = {1'b0, RgB} + {1'b0, ~RgA} + RgCI;
57             CO_REF = temp[WIDTH];
58             VO_REF = (RgB[WIDTH-1] != RgA[WIDTH-1]) &&
59                 (temp[WIDTH-1] != RgB[WIDTH-1]);
60         end
61         2'b11: temp = {1'b0, RgA | RgB};
62     endcase
63
64     F_REF = temp[WIDTH-1:0];
65     NO_REF = F_REF[WIDTH-1];
66     ZO_REF = (F_REF == {WIDTH{1'b0}});
67     NORM_RIGHT_REF = VO_REF && (F_REF[WIDTH-1] != F_REF[WIDTH-2]);
68
69     if (NORM_RIGHT_REF) begin
70         F_NORM_REF = {F_REF[WIDTH-1], F_REF[WIDTH-1:1]};
71     end else begin
72         F_NORM_REF = F_REF;
73     end
74 end
75 endmodule

```

sim/stimulus.v

```

1  `timescale 1 ns / 1 ps
2
3  module stimulus
4  #(
5      parameter WIDTH = 12
6  )
7  (
8      output reg [WIDTH-1:0] A_ALB,

```

```

9      output reg [WIDTH-1:0] B_ALB,
10     output reg CI,
11     output reg [1:0] ALB_MI,
12     input clock,
13     input resetb
14 );
15     task apply_vector;
16         input [1:0] mi;
17         input [WIDTH-1:0] a;
18         input [WIDTH-1:0] b;
19         input ci;
20         begin
21             @(negedge clock);
22             ALB_MI = mi;
23             A_ALB = a;
24             B_ALB = b;
25             CI = ci;
26         end
27     endtask
28
29     initial begin
30         A_ALB = {WIDTH{1'b0}};
31         B_ALB = {WIDTH{1'b0}};
32         CI = 1'b0;
33         ALB_MI = 2'b00;
34
35         wait (resetb == 1'b0);
36         wait (resetb == 1'b1);
37
38         apply_vector(2'b00, 12'b0000000001010, 12'b0000000000101, 1'b0);
39         apply_vector(2'b11, 12'b0000000001010, 12'b0000000000101, 1'b1);
40
41         apply_vector(2'b01, 12'b0000000000001, 12'b0000000000001, 1'b0);
42         apply_vector(2'b01, 12'b0000000000000, 12'b0000000000000, 1'b0);
43         apply_vector(2'b01, 12'b0110001110, 12'b1010010111, 1'b0);
44         apply_vector(2'b01, 12'b000110001110, 12'b001010010111, 1'b0);
45         apply_vector(2'b01, 12'b1101101000, 12'b111001110001, 1'b1);
46         apply_vector(2'b01, 12'b011000000000, 12'b001100000000, 1'b0);
47         apply_vector(2'b01, 12'b100100000000, 12'b101000000000, 1'b0);
48
49         apply_vector(2'b10, 12'b0000000000001, 12'b0000000000011, 1'b1);
50         apply_vector(2'b10, 12'b0000000000011, 12'b0000000000001, 1'b1);
51         apply_vector(2'b10, 12'b001010010111, 12'b000110001110, 1'b1);
52         apply_vector(2'b10, 12'b000110001110, 12'b110101101001, 1'b1);
53
54         apply_vector(2'b01, 12'b111111111111, 12'b0000000000001, 1'b0);
55         apply_vector(2'b00, 12'b101010101010, 12'b010101010101, 1'b0);
56
57         repeat (8) @(posedge clock);
58         $finish;
59     end
60 endmodule

```

sim/checker.v

```

1  `timescale 1 ns / 1 ps
2
3  module checker
4  #(
5      parameter WIDTH = 12
6  )
7  (
8      input [WIDTH-1:0] F_ALB,
9      input [WIDTH-1:0] F_NORM,
10     input CO,
11     input VO,
12     input NO,
13     input ZO,
14     input NORM_RIGHT,
15     input [WIDTH-1:0] F_REF,
16     input [WIDTH-1:0] F_NORM_REF,
17     input CO_REF,
18     input VO_REF,
19     input NO_REF,
20     input ZO_REF,
21     input NORM_RIGHT_REF,
22     input clock,
23     input resetb
24 );
25     integer errors;
26
27     initial begin
28         errors = 0;
29     end
30
31     always @(negedge clock) begin
32         if (resetb == 1'b1) begin
33             if ({CO, VO, NO, ZO, NORM_RIGHT, F_ALB, F_NORM} ==
34                 {CO_REF, VO_REF, NO_REF, ZO_REF, NORM_RIGHT_REF, F_REF, F_NORM_REF})
35                 → begin
36                     $display("%0t correct F=%b FN=%b C=%b V=%b N=%b Z=%b NR=%b",
37                             $time, F_ALB, F_NORM, CO, VO, NO, ZO, NORM_RIGHT);
38                 end else begin
39                     errors = errors + 1;
40                     $display("%0t error dut=%b_%b_%b_%b_%b_%b_%b
41                             → ref=%b_%b_%b_%b_%b_%b_%b",
42                             $time, CO, VO, NO, ZO, NORM_RIGHT, F_ALB, F_NORM,
43                             CO_REF, VO_REF, NO_REF, ZO_REF, NORM_RIGHT_REF,
44                             F_REF, F_NORM_REF);
45                 end
46             end
47         end
48     end
49
50     initial begin
51         #220;
52         if (errors == 0) begin
53             $display("TEST PASSED");
54         end else begin

```

```

52         $display("TEST FAILED: %0d errors", errors);
53     end
54 end
55 endmodule

```

sim/top.v

```

1  `timescale 1 ns / 1 ps
2
3  module top;
4      parameter WIDTH = 12;
5
6      wire [WIDTH-1:0] A_ALB;
7      wire [WIDTH-1:0] B_ALB;
8      wire [WIDTH-1:0] F_ALB;
9      wire [WIDTH-1:0] F_NORM;
10     wire [WIDTH-1:0] F_REF;
11     wire [WIDTH-1:0] F_NORM_REF;
12     wire CI;
13     wire [1:0] ALB_MI;
14     wire CO;
15     wire VO;
16     wire NO;
17     wire ZO;
18     wire NORM_RIGHT;
19     wire CO_REF;
20     wire VO_REF;
21     wire NO_REF;
22     wire ZO_REF;
23     wire NORM_RIGHT_REF;
24     reg clock;
25     reg resetb;
26
27     alb #(WIDTH) dut (
28         .F_ALB(F_ALB),
29         .F_NORM(F_NORM),
30         .CO(CO),
31         .VO(VO),
32         .NO(NO),
33         .ZO(ZO),
34         .NORM_RIGHT(NORM_RIGHT),
35         .A_ALB(A_ALB),
36         .B_ALB(B_ALB),
37         .CI(CI),
38         .ALB_MI(ALB_MI),
39         .clock(clock),
40         .resetb(resetb)
41     );
42
43     alb_ref #(WIDTH) ref_model (
44         .F_REF(F_REF),
45         .F_NORM_REF(F_NORM_REF),
46         .CO_REF(CO_REF),
47         .VO_REF(VO_REF),

```

```

48     .NO_REF(NO_REF),
49     .ZO_REF(ZO_REF),
50     .NORM_RIGHT_REF(NORM_RIGHT_REF),
51     .A_ALB(A_ALB),
52     .B_ALB(B_ALB),
53     .CI(CI),
54     .ALB_MI(ALB_MI),
55     .clock(clock),
56     .resetb(resetb)
57 );
58
59 stimulus #(WIDTH) stim (
60     .A_ALB(A_ALB),
61     .B_ALB(B_ALB),
62     .CI(CI),
63     .ALB_MI(ALB_MI),
64     .clock(clock),
65     .resetb(resetb)
66 );
67
68 checker #(WIDTH) check (
69     .F_ALB(F_ALB),
70     .F_NORM(F_NORM),
71     .CO(CO),
72     .VO(VO),
73     .NO(NO),
74     .ZO(ZO),
75     .NORM_RIGHT(NORM_RIGHT),
76     .F_REF(F_REF),
77     .F_NORM_REF(F_NORM_REF),
78     .CO_REF(CO_REF),
79     .VO_REF(VO_REF),
80     .NO_REF(NO_REF),
81     .ZO_REF(ZO_REF),
82     .NORM_RIGHT_REF(NORM_RIGHT_REF),
83     .clock(clock),
84     .resetb(resetb)
85 );
86
87 initial begin
88     clock = 1'b0;
89     forever #5 clock = ~clock;
90 end
91
92 initial begin
93     resetb = 1'b0;
94     #20 resetb = 1'b1;
95 end
96 endmodule

```

sim/run.do

```

1 transcript on
2 if {[file exists work/_info]} {

```

```

3     echo "Reusing existing work library"
4 } elseif {[file exists work]} {
5     echo "Using existing work directory"
6 } else {
7     vlib work
8 }
9 vlog ../src/alb.v alb_ref.v stimulus.v checker.v top.v
10 vsim work.top
11 add wave -radix binary sim:/top/clock
12 add wave -radix binary sim:/top/resetb
13 add wave -radix binary sim:/top/ALB_MI
14 add wave -radix binary sim:/top/CI
15 add wave -radix binary sim:/top/A_ALB
16 add wave -radix binary sim:/top/B_ALB
17 add wave -radix binary sim:/top/F_ALB
18 add wave -radix binary sim:/top/F_NORM
19 add wave -radix binary sim:/top/CO
20 add wave -radix binary sim:/top/VO
21 add wave -radix binary sim:/top/NO
22 add wave -radix binary sim:/top/ZO
23 add wave -radix binary sim:/top/NORM_RIGHT
24 run -all

```

sim/wave_pretty.do

```

1 transcript on
2
3 if {[file exists work/_info]} {
4     echo "Reusing existing work library"
5 } elseif {[file exists work]} {
6     echo "Using existing work directory"
7 } else {
8     vlib work
9 }
10
11 vlog ../src/alb.v alb_ref.v stimulus.v checker.v top.v
12 vsim work.top
13
14 delete wave *
15
16 add wave -divider "Clock and reset"
17 add wave -radix binary sim:/top/clock
18 add wave -radix binary sim:/top/resetb
19
20 add wave -divider "Control"
21 add wave -radix binary sim:/top/ALB_MI
22 add wave -radix binary sim:/top/CI
23
24 add wave -divider "Operands"
25 add wave -radix binary sim:/top/A_ALB
26 add wave -radix binary sim:/top/B_ALB
27
28 add wave -divider "ALB result"
29 add wave -radix binary sim:/top/F_ALB

```

```

30 add wave -radix binary sim:/top/F_NORM
31
32 add wave -divider "Reference result"
33 add wave -radix binary sim:/top/F_REF
34 add wave -radix binary sim:/top/F_NORM_REF
35
36 add wave -divider "Flags"
37 add wave -radix binary sim:/top/CO
38 add wave -radix binary sim:/top/VO
39 add wave -radix binary sim:/top/NO
40 add wave -radix binary sim:/top/ZO
41 add wave -radix binary sim:/top/NORM_RIGHT
42
43 add wave -divider "Reference flags"
44 add wave -radix binary sim:/top/CO_REF
45 add wave -radix binary sim:/top/VO_REF
46 add wave -radix binary sim:/top/NO_REF
47 add wave -radix binary sim:/top/ZO_REF
48 add wave -radix binary sim:/top/NORM_RIGHT_REF
49
50 configure wave -namecolwidth 180
51 configure wave -valuecolwidth 120
52 configure wave -justifyvalue left
53 configure wave -signalnamewidth 1
54 configure wave -timelineunits ns
55 configure wave -gridoffset 0
56 configure wave -gridperiod 10
57 configure wave -griddelta 40
58
59 run -all
60 wave zoom full

```

Висновок

У ході виконання домашньої контрольної роботи було розроблено параметризований АЛБ відповідно до варіанта, визначеного за номером залікової книжки 4106. Реалізовано мікрооперації $R \vee S$, $R + S + CI$ та $S - R - 1 + CI$, передбачено вхідні буферні регістри, сформовано ознаки CO, VO, NO, ZO.

Для перевірки створено тестовий рівень, що складається з генератора вхідних даних, еталонної математичної моделі та модуля автоматичної перевірки. Функціональне моделювання в ModelSim підтвердило відповідність результатів АЛБ еталонній моделі. Додатково реалізовано постпроцесор нормалізації, який за потреби виконує зсув результату на один розряд праворуч.